ASSIGNMENT 5 – PART B – TASK B4

Final Engineering Decision

Programming in Java | CO5 – Data Persistence Engineering

Task scope: Implement a file-to-database migration with validation reporting, inspect JDBC database metadata, and evaluate the production storage approach for 12 campuses and concurrent counters.

1. B4.1 – Practical migration strategy

The Java program reads the existing five-field pass file line by line using BufferedReader. Each line is split into five fields and validated before it is added to the migration list. Required text fields must not be blank, route number must be a positive integer, amount must be numeric and non-negative, and duplicate pass IDs in the same input are rejected.

Rejected rows are written to output/validation_failures.csv with the source line number, original record and reason. Valid records are inserted with a parameterized PreparedStatement in batches of 1,000. Auto-commit is disabled; all batches are committed together. A database exception triggers rollback, preventing a partially completed migration.

Input schema: pass_id, reg_no, name, route_no, amount. The B4 database table adds campus_code so that the primary key is (campus_code, pass_id), allowing pass IDs to be scoped to a campus.

2. B4.2 – Database metadata inspection

DatabaseMetaData is obtained from the active JDBC Connection. The program prints the database product name and version, JDBC driver name and version, and the table names visible in the current catalog. This avoids hard-coding environment assumptions and helps identify driver, server or schema differences when deploying to another server.

3. B4.3 – Production decision

Decision: use a shared relational database. Twelve campuses and multiple counters require safe concurrent writes, consistent pass identifiers, indexed searching, transactions and operational backup/access controls. A relational database provides primary-key constraints, transactions, indexes and concurrency control. Text files are easy to exchange but require custom locking and scanning. Java serialization is Java-specific and is not a suitable shared multi-user database.

Alternative: separate serialized file per campus

If each campus maintains its own serialized file, a global search must inspect all 12 files. If campus i stores n■ records, total search work is O(Σn■) = O(n). Loading all campus data into memory requires O(n) space; processing one campus file at a time reduces working memory but still visits all files. Concurrent counters require file locks or a single-writer service per campus. Cross-campus updates and uniqueness require additional coordination, making this design more complex than a shared database.

4. Complexity and engineering considerations

Operation	Expected complexity / consideration
Validation pass over m input rows	O(m) time; O(v) memory for v accepted records in this implementation
Batch migration	O(v) row processing; network round trips reduced by batches of up to 1,000
Indexed pass lookup	Typically O(log n) index traversal; actual performance depends on engine and index
Search across campus serialized files	O(Σn■) = O(n) for a full scan across all campuses
All files loaded in memory	O(n) memory; streaming one campus at a time reduces working memory

5. Execution and evidence checklist

• Create assignment5_b4 and bus_pass by running sql/create_database_and_table.sql.

• Add MySQL Connector/J to lib/ and configure DB_URL, DB_USER and DB_PASSWORD.

• Compile and run B4Migration with input/passes_b4.csv and a campus code.

• Verify output/validation_failures.csv and the printed accepted/rejected/inserted counts.

• Verify database product, driver version and table list from DatabaseMetaData.

• Run the SELECT statements in README.md and capture screenshots of the actual results.

Important: No database timings or successful runtime output are fabricated in this report. The Java source was compile-checked; actual JDBC execution requires a local MySQL server and Connector/J. Insert your own execution evidence and measurements before submission.